Instruction-based Time Series Editing

Jiaxing Qiu
University of Virginia
Charlottesville, VA, USA
jq2uw@virginia.edu

Dongliang Guo
University of Virginia
Charlottesville, VA, USA
ktm8eh@virginia.edu

Brynne Sullivan
University of Virginia
Charlottesville, VA, USA
bsa4m@uvahealth.org

Teague R. Henry
University of Virginia
Charlottesville, VA, USA
ycp6wm@virginia.edu

Thomas Hartvigsen
University of Virginia
Charlottesville, VA, USA
hartvigsen@virginia.edu

Abstract

In time series editing, we aim to modify some properties of a given time series without altering others. For example, when analyzing a hospital patient's blood pressure, we may add a sudden early drop and observe how it impacts their future, while preserving other conditions. Existing diffusion-based editors rely on rigid, predefined attribute vectors as conditions and produce all-or-nothing edits through sampling. This attribute- and sampling-based approach limits flexibility in condition format and lacks customizable control over editing strength. To overcome these limitations, we introduce Instruction-based Time Series Editing, where users specify intended edits using natural language. This allows users to express a wider range of edits in a more accessible format. We then introduce InstructTime, the first instruction-based time series editor. InstructTime takes in time series and instructions, embeds them into a shared multi-modal representation space, then decodes their embeddings to generate edited time series. By learning a structured multi-modal representation space, we can easily interpolate between embeddings to achieve varying degrees of edit. To handle local and global edits together, we propose multi-resolution encoders. In our experiments, we use synthetic and real datasets and find that InstructTime is a state-of-the-art time series editor: InstructTime achieves high-quality edits with controllable strength, can generalize to unseen instructions, and can be easily adapted to unseen conditions through few-shot learning.

CCS Concepts

• **Computing methodologies** → *Modeling methodologies*; **Learning latent representations**.

Keywords

Time Series Editing; Natural Language Instruction; Shared Embedding Space; Contrastive Learning

ACM Reference Format:
Jiaxing Qiu, Dongliang Guo, Brynne Sullivan, Teague R. Henry, and Thomas Hartvigsen. 2026. Instruction-based Time Series Editing. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780299

This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780299

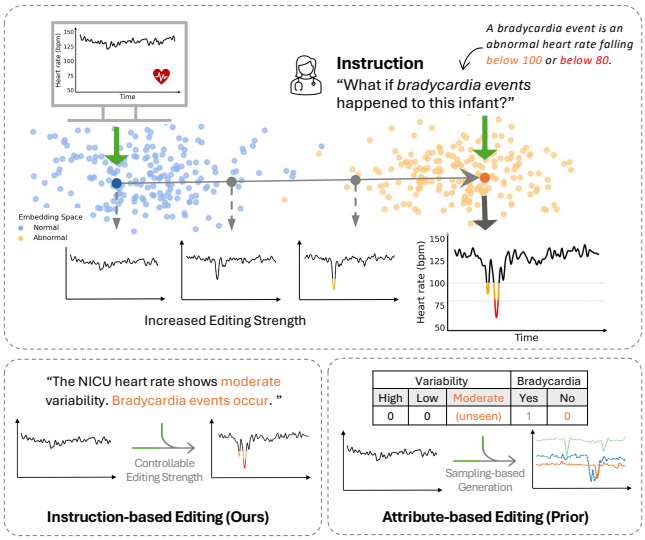


Figure 1: ***Instruction-based time series editing*** **modifies a given time series using natural language instructions (e.g., adding abnormal events to a normal heart rate). Our approach enables controllable editing strength and generalizes to unseen instructions. In contrast, existing attribute- and sampling-based methods edit time series with uncontrolled strength based on predefined attribute vectors.**

Resource Availability:
The source code of this paper has been made publicly available at https://github.com/JiaxingQiu/InstructTime.

1 Introduction

Many recent works have focused on time series generation [2, 2, 7, 14, 15, 44, 46, 49, 54, 62, 66, 67] as a way to synthesize realistic data for time series analysis tasks such as classification [28, 59, 63], forecasting [3, 9, 48], and counterfactual reasoning [3, 5, 29, 31]. A recent extension is Time Series Editing (TSE), which enables fine-grained manipulation of an input time series by directly modifying it to match desired conditions while preserving the characteristics of the original data [27, 39, 68]. TSE differs from the time series generation task in that it modifies an input time series toward a target condition rather than generating a completely new time series from scratch [27]. For example, as illustrated in Figure 1, a user may wonder "given a normal heart rate time series, what would happen

if my patient's heart rate rapidly falls below 80 beats per minute?" This describes *bradycardia*, a dangerous situation for hospitalized infants [1]. A successful time series editor would take in the normal heart rate and new condition (bradycardia happened), then generate a new time series showing the likely locations, severity, and shapes of such abnormalities if they were to occur. However, no existing methods can handle natural language edits.

State-of-the-art TSE methods focus on attribute-level control and use conditional diffusion [27, 39]. However, these methods have two limitations. Firstly, they represent attributes as continuous or categorical feature vectors. These attributes require handcrafted feature engineering, and constrain all future edits to capture only these attributes, and limits users to this hardcoded interaction with the editor. Secondly, diffusion-based methods perform all-or-nothing edits by sampling diverse outputs, offering no control over editing strength. In contrast, allowing edits with varying condition influence enables finer user control and supports insight and hypothesis generation—e.g., gradually adding abnormality to a normal heart rate from mild to severe.

We propose a new time series editing task: **Instruction-based Time Series Editing**. An editor takes in a time series and a text instruction specifying target conditions and generates a modified time series that reflects the target conditions. Instead of hard-coding attributes, users can directly guide time series editing using natural language instructions, which flexibly integrate structured and unstructured, qualitative and quantitative conditions into a unified semantic representation. In real-world settings, text paired with time series—such as nurse notes or patient narratives—can capture nuanced, individualized conditions under which the time series is generated. Instruction-based time series editing addresses this setting by enabling editors to condition on heterogeneous free-text inputs, where condition factors vary widely across instances.

Instruction-based time series editing poses three key challenges. First, an editor should handle multiple conditions described in a single natural language instruction and fuse them with a time series exhibiting multi-resolution patterns—some conditions affect global patterns such as trends and seasonality, while others lead to localized changes such as abrupt mean shifts or abnormalities. Secondly, since conditions are expressed discretely in natural language [52], fine-grained control over editing strength is especially necessary. For example, rather than editing a heart rate time series from "normal" to "abnormal", the user may progressively add abnormality at mild, moderate, and severe levels during editing. To allow such interpolated editing, the editor must encode the varying degrees to which an instruction can influence a time series. Lastly, the editor should leverage the richness of natural language and generalize to new expressions of known conditions or to unseen conditions, based on their semantic relationships to observed ones.

To overcome these challenges, we introduce **InstructTime**, the first instruction-based time series editing method. InstructTime maps time series and text instructions into a shared unit-length hypersphere via contrastive learning [42], and decodes linearly-interpolated embeddings of a time series and its target instruction into new time series that satisfy the instructed conditions to varying degrees. We evaluate InstructTime on both synthetic and real-world datasets. Under instruction-based settings, our model achieves the state-of-the-art performance, while enabling interpolated editing and generalizability to unseen instructions with few-shot tuning. We also evaluate a categorical attribute version of InstructTime to fairly compare with recent non-instruction-based time series editors, and find that it achieves higher fidelity and comparable performance to state-of-the-art methods.

In summary, our contributions are as follows:

(1) We introduce instruction-based time series editing, a novel problem setting that lets users edit time series using natural language instructions.
(2) We propose InstructTime, the first instruction-based time series editor, which enables multi-resolution time series editing via multi-condition instructions, enabling high-fidelity editing with controllable strength.
(3) Our experiments show InstructTime successfully enables instruction-based editing, achieves state-of-the-art performance, and generalizes to unseen instructions.

2 Related Works

Time-Series Generation. The generation of time-series data has progressed through advancements in both unconditional and conditional generative models. *Unconditional generation* focuses on producing time-series data without explicit constraints. Early approaches employed generative adversarial networks [49, 62, 66] and diffusion models [7, 44, 54] to synthesize realistic sequences. Unconditional models lack the ability to guide the generation toward desired outcomes. *Conditional generation* provides finer control over the generated time series by conditioning on external signals. Early efforts explored global attribute control via extensions of GANs and VAEs [2, 14, 66]. More recent developments introduced hierarchical frameworks [57] and attention-based architectures [34], enabling more granular control. Diffusion-based conditional models have emerged [12, 19, 39, 56], offering improved controllability and sample diversity. Several recent works have used language prompts to generate synthetic time series [18, 24, 30]. Nonetheless, these methods focus on generating new time series from scratch without editing an input, thus are not directly comparable to the editing task we consider.

Time Series Editing. Time series editing differs from the time series generation task in that it modifies *an input time series* to satisfy target conditions while preserving the underlying structure of the original data as much as possible. It enables more fine-grained manipulation while retaining the specific patterns of the input. In contrast, a conditional time series generation model takes only a condition as input and generate a new time series from scratch. It samples from the target conditional distribution without the need to preserve any properties of an existing time series. Recent approaches build upon conditional diffusion models with enhancements such as multi-resolution encoding [27, 39]. TEdit [27] extends TimeWeaver [39], a conditional diffusion-based generation method, into an editing framework by introducing a generation process that uses a denoising diffusion sampler (e.g., the denoising diffusion implicit model [50]) along with a multi-resolution architecture. It takes as input a triplet of an input time series, its original attribute vector, and the target attribute vector, and generates an edited time series where the attributes that differ from the original are modified, while the remaining ones are preserved. These methods require



Figure 2: InstructTime architecture, training, and interpolated editing. InstructTime includes a multi-resolution time series encoder, an instruction encoder, and a time series conditional decoder. During training (left), time series–description pairs are encoded into latent representations z_x and z_y in a shared hyperspherical space, and the decoder reconstructs the input time series from both. During editing (right), we encode a time series and an instruction onto the shared embedding space, then decode interpolated embeddings between them to generate edits at varying editing strength.

Table 1: Differences in problem definitions of related tasks.

Task	Input		Output	Objectives	Methods
	TS	Condition	TS		
Conditional Time Series Generation	✗	✓	✓	(1) generate a new time series from scratch, following target distribution.	[2, 12, 14, 18, 19, 24, 30, 56, 66]
Time Series–Text Fusion	✓	✓	✗	(1) representation learning.	[6, 8, 20, 22, 23, 33, 36, 64, 70, 71]
Time Series Editing	✓	✓ (Attributes)	✓	(1) *editability*: edit a given time series towards target conditions;	TEdit [27], TimeWeaver [27, 39]
	✓	✓ (Instruction)	✓	(2) *preservability*: preserve other conditions of the input time series.	**InstructTime (Ours)**

conditions to be categorical or continuous attribute vectors, limiting generalizability to new conditions during inference. Moreover, while diffusion-based editors can produce diverse outputs through stochastic sampling, they lack mechanisms to explicitly regulate the degree of editing. In this study, we take a step further by allowing users to edit time series using natural language instructions and controllable editing strength.

Contrastive Learning for Time Series–Language Fusion. Contrastive learning has emerged as a powerful paradigm for aligning heterogeneous modalities in a shared embedding space. Pioneered by CLIP [42] for image–language alignment, contrastive methods have since been extended to time series for tasks such as retrieval, classification, and forecasting. Recent works [6, 8, 20, 22] show that time series can be effectively aligned with textual descriptions or labels via contrastive objectives, enabling cross-modal retrieval and classification. To learn better time series representations, several studies [33, 36, 71] use synthetic descriptions or structured metadata to supervise alignment, while others [23, 64, 70] align future trends with language cues to improve temporal generalization. While these efforts demonstrate the effectiveness of contrastive learning in fusing time series and language, they are largely limited to static latent representation alignment and are only evaluated on retrieval, classification, or forecasting tasks. A detailed comparison on problem definitions of the related tasks is provided in Table 1.

3 Instruction-based Time Series Editing

3.1 Problem Formulation

Let $\mathbf{x} \in \mathbb{R}^T$ be a time series with T timesteps, and let $\mathbf{c} = [c_1, c_2, \ldots, c_L]$ be a text instruction consisting of L tokens, describing the target condition of the edited time series. The target condition refers to the realization of a set of K attributes on the target time series, $\tilde{\mathbf{c}} = \{\tilde{c}_i\}_{i=1}^{K}$, some of which may differ from those of the input time series, while others remain the same.

Definition 1 (Instruction-based Time Series Editing). *Given an input time series* $\mathbf{x}$ *and a natural language instruction* $\mathbf{c}$*, the instruction-based time series editing task is to build a function* f_θ *that modifies* $\mathbf{x}$ *into a new time series* $\hat{\mathbf{x}} = f_\theta(\mathbf{x}, \mathbf{c})$*, such that* $\hat{\mathbf{x}}$ *reflects the target condition described in the instruction* $\mathbf{c}$*.*

Compared to prior work [27], we use a text instruction $\mathbf{c}$ instead of a feature vector $\tilde{\mathbf{c}}$ to guide the editing. We also relax prior problem definition by **not requiring the original condition of x as an additional input**, because the model should be expected to infer the original condition directly from $\mathbf{x}$ without relying on external manual specification. An effective instruction-based editor should also parse multiple conditions described in the instruction and modify the subset that differ from the input's conditions while preserving those that align—thereby achieving both editability and preservability [27]—in the edited time series.

In this section, the terms "description" and "instruction" are both used to denote textual inputs. We use "description" to refer to the text paired with each time series that is fused with the time series during training, while "instruction" indicates the user-specified input at inference time that guides the time series editing process.

3.2 InstructTime

We propose InstructTime, the first instruction-based time series editor, leveraging CLIP-based contrastive learning and a novel interpolated editing procedure for controllable editing strength. As illustrated in Figure 2 (left), InstructTime combines a multi-resolution time series encoder, an instruction encoder generalizable to diverse

semantic expressions of conditions, and a time series decoder that generates time series by modeling both intra- and inter-modality relationships via a self-attention mechanism.

Multi-resolution time series encoder $\mathcal{E}_\phi$. The time series encoder $\mathcal{E}_\phi$ transforms a time series $\mathbf{x}$ with T timesteps into a latent embedding $\mathbf{z}_x$ in the D-dimensional unit-length hypersphere,

$$\mathbf{z}_x = \mathcal{E}_\phi(\mathbf{x}) \quad \text{s.t. } \|\mathbf{z}_x\|_2 = 1.$$

A key challenge in encoding a time series is capturing multi-resolution features of the input $\mathbf{x}$, as meaningful patterns can occur at different temporal scales—ranging from global trends (e.g., accelerating heart rate) to local fluctuations (e.g., abrupt drops in heart rate). To effectively embed information across multiple resolutions, we use a convolutional neural network (CNN)-based encoder $\mathcal{E}_\phi$ with varying kernel sizes (Figure 2). Such CNN-based multi-resolution design has proven effective in both time series analysis tasks [11, 27, 38, 40, 61] and multi-scale image segmentation and classification [21, 32, 65, 69]. $\mathcal{E}_\phi$ consists of k parallel 1D CNNs, each with a kernel size proportional to T. Varying kernel sizes capture patterns at different temporal resolutions, with larger kernels capturing global patterns such as trends and seasonality, while smaller kernels are more effective at detecting local variations such as abrupt shifts or outliers. Each CNN outputs a d-dimensional vector $\mathbf{z}_{x_k}$, capturing a latent representation of the time series at a distinct temporal resolution. The k vectors are then concatenated to form the final D-dimensional time series embedding, $\mathbf{z}_x = [\mathbf{z}_{x_1} \,\|\, \mathbf{z}_{x_2} \,\|\, \cdots \,\|\, \mathbf{z}_{x_k}]$, where $D = k \cdot d$. Lastly, $\mathbf{z}_x$ is normalized to have unit length, constraining the embedding space to a hypersphere and preventing uncontrolled distances between time series in an unbounded space [13].

Regarding the architecture, each CNN consists of a 1D convolutional layer with stride = 1 and zero-padding set to half of the kernel size to preserve temporal length. Each CNN is followed by a ReLU activation, batch normalization, a max-pooling layer with stride = 2, and a linear projection to the d-dimensional embedding space. The design choices for k and the fractions that determine kernel sizes proportional to T depend on the datasets and experimental settings, and are therefore described in the implementation details of the experimental setup (Section 4.1).

Instruction encoder $\mathcal{E}_\theta$. The text encoder $\mathcal{E}_\theta$ encodes the instruction $\mathbf{c}$ into an embedding $\mathbf{z}_c$, which also lies in the D-dimensional unit-length hypersphere,

$$\mathbf{z}_c = \mathcal{E}_\theta(\mathbf{c}) \quad \text{s.t. } \|\mathbf{z}_c\|_2 = 1.$$

To efficiently compress an instruction into a semantic embedding, we first leverage a pretrained text embedding model (frozen parameters) to encode the instruction $\mathbf{c}$ into a numeric vector. Such vector captures rich semantic meaning and paraphrastic similarity of the multiple conditions described in the instruction [26, 45]. The vector is then processed by k parallel multi-layer perceptrons (MLPs), each predicting a d-dimensional vector $\mathbf{z}_{c_k}$. These vectors are concatenated to form the final D-dimensional instruction embedding, $\mathbf{z}_c = [\mathbf{z}_{c_1} \,\|\, \mathbf{z}_{c_2} \,\|\, \cdots \,\|\, \mathbf{z}_{c_k}]$, where $D = k \cdot d$. This chunked representation enables resolution-wise semantic alignment—where the rth segment of the instruction embedding aligns with the rth segment of the time series embedding ($r = 1, \ldots, k$)—thereby linking each segment's semantic representation to its corresponding temporal resolution during both contrastive fusion and decoding. A detailed explanation of how the contrastive learning encourages resolution-wise semantic alignment is provided in Appendix B. Lastly, $\mathbf{z}_c$ is also normalized to have unit length to ensure it resides in the same bounded embedding space as $\mathbf{z}_x$.

Regarding the architecture, each MLP consists of LayerNorm, a two-layer feedforward network with GELU activations and a linear projection to the d-dimensional embedding space.

Conditional time series decoder Ψ. Given a time series embedding $\mathbf{z}_x$ and an instruction embedding $\mathbf{z}_c$, the goal of the decoder Ψ is to generate an edited time series $\hat{\mathbf{x}}$ of length T that reflects the target condition encoded in $\mathbf{z}_c$ while preserving the unique characteristics of the input time series encoded in $\mathbf{z}_x$,

$$\hat{\mathbf{x}} = \Psi(\mathbf{z}_x, \mathbf{z}_c).$$

Our time series decoder Ψ leverages the self-attention mechanism from the Transformer architecture [58], allowing it to model complex dependencies between the time series and instruction embeddings. It takes the time series and instruction embeddings as a token sequence of length 2, applies positional encoding, and processes them through eight attention-based blocks, each consisting of multi-head self-attention, feedforward layers, and residual connections. The self-attention mechanism allows each token to attend to both itself and other tokens. This enables the model to capture relationships within the time series embedding (e.g., between multi-resolution representations) and within the instruction embedding (e.g., between multiple conditions). It also models the relationships between the two modalities with respect to the mappings between the multi-resolution representations of time series and the multiple semantic conditions. Lastly, the output hidden state corresponding to the time series token is passed through a linear head to generate a time series $\hat{\mathrm{x}}$ of the same length T as the input time series x.

Training. The goal of training is to learn a shared embedding space where time series and instruction embeddings align semantically, while accurately reconstructing the original time series from both modalities. InstructTime can be trained on datasets containing pairs of time series and corresponding descriptions $\{(\mathrm{x}_i, \mathrm{c}_i)\}_{i=1}^N$, where N denotes the sample size, by minimizing the following loss function:

$$\mathcal{L} = \mathcal{L}_{\text{contrast}}(\mathrm{z_x}, \mathrm{z_c}) + \alpha \cdot \mathcal{L}_{\text{recon}}(\mathrm{x}, \hat{\mathrm{x}});$$

$$\mathcal{L}_{\text{contrast}} = \tfrac{1}{2N} \textstyle\sum_{i=1}^N [-\log\left(\frac{\exp(\mathrm{z_x}^{(i)} \cdot \mathrm{z_c}^{(i)})}{\sum_{j=1}^N \exp(\mathrm{z_x}^{(i)} \cdot \mathrm{z_c}^{(j)})}\right) - \log\left(\frac{\exp(\mathrm{z_c}^{(i)} \cdot \mathrm{z_x}^{(i)})}{\sum_{j=1}^N \exp(\mathrm{z_c}^{(i)} \cdot \mathrm{z_x}^{(j)})}\right)],$$

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^N \|\mathrm{x}_i - \hat{\mathrm{x}}_i\|_2^2 .$$

The contrastive loss $\mathcal{L}_{\text{contrast}}$ is the symmetric InfoNCE loss used in CLIP [42] and the reconstruction loss $\mathcal{L}_{\text{recon}}$ is the mean squared error (MSE) between x and its reconstruction $\hat{\mathrm{x}}$. We train the model in two steps: (1) we first minimize $\mathcal{L}_{\text{contrast}}$ to update the parameters of $\mathcal{E}_\phi$ and $\mathcal{E}_\theta$, encouraging paired embeddings $\mathrm{z_x}$ and $\mathrm{z_c}$ to move closer on the unit-length D-dimensional hypersphere; (2) we then jointly minimize $\mathcal{L}_{\text{contrast}}$ and $\mathcal{L}_{\text{recon}}$ through the overall loss $\mathcal{L}$ to update the parameters of both encoders and the decoder Ψ. This allows the model to reconstruct the time series under the constraint that the time series and instruction embeddings reside closely in a shared embedding space. In step (2), to prevent the model from

de-prioritizing contrastive learning, the balancing weight α controls the unit contribution of the reconstruction loss relative to the contrastive loss. The training configurations in our experiments on various datasets are described in Section 4.1 *training details*.

3.3 Interpolated Editing Procedure

To control editing strength at inference, we use an interpolated editing procedure (Figure 2, right). It is motivated by our embedding geometry and encoder–decoder design: the shared encoder places similar time series close together and distinct ones farther apart in the unit-length subspace, creating geometric continuity that bridges discrete textual conditions. For instance, time series with large positive slopes ("strongly upward") lie near those with small positive ("weakly upward") and small negative ("weakly downward") slopes, forming a smooth semantic continuum from upward to downward. Interpolation between input time series and instruction embeddings thus traces a smooth transition between their conditions. Similar interpolation procedures have been used in text [4], image [43, 60], and motion [55] generation tasks.

Given a time series and an instruction, their embeddings z_x and z_c are projected onto a shared unit-length D-dimensional hypersphere. We linearly interpolate between z_x and z_c with weight $w \in [0, 1]$ and normalize the embedding to obtain the interpolated embedding z_w. The decoder Ψ then maps z_w to a time series that gradually transitions toward the target condition as w increases.

$$z_w = \frac{(1-w)\, z_x + w\, z_c}{\|(1-w)\, z_x + w\, z_c\|}, \qquad \hat{x}_w = \Psi(z_w, z_c).$$

Larger w result in stronger editing, making the target condition more prominent in the edited time series, whereas smaller w produce time series that more closely resemble the input time series while moderately reflecting the target condition. For example, editing a downward trend toward an upward one, increasing w from 0 to 1 shifts the trend from downward to flat, then to upward. When $w = 0$, the decoder reconstructs the input: $\hat{x}_{w=0} = \Psi(z_x, z_c)$. When $w = 1$, the decoder generates a new time series solely from text instruction: $\hat{x}_{w=1} = \Psi(z_c, z_c)$. This means InstructTime supports both time series editing and conditional generation. We also experimented with spherical linear interpolation for unit-norm hypersphere geometry. We find no notable performance difference—due to tightly clustered embeddings—and greater numerical stability for normalized linear interpolation for nearly parallel embeddings.

3.4 Few-shot Tuning on Unseen Conditions

Editing time series via natural language enables generalization to rich semantic expressions, including (1) novel phrasings of known conditions and (2) instructions for entirely unseen conditions. In (1), a pretrained text encoder helps $\mathcal{E}_\theta$ map new expressions close to seen ones in embedding space, enabling zero-shot generalization. In (2), however, it's a strong assumption that unseen instructions align with their (also unseen) time series in embedding space, and that Ψ can decode them correctly in a zero-shot manner. To enable data-efficient tuning on conditions unseen during training, we also introduce a few-shot tuning procedure. Assuming users can provide a small number of time series–instruction pairs from the unseen condition, we adopt a two-step approach: first, edit each new time series toward seen conditions using varying interpolation weights (e.g., 0.1 to 0.9) to create synthetic examples paired with seen instructions; then, tune model parameters on the combined set of synthetic and unseen pairs by minimizing the loss $\mathcal{L}$.

4 Experiments

In this section, we evaluate InstructTime on three key capabilities—editing quality, interpolated editing, and generalizability to unseen instructions. We use both a synthetic dataset where time series are associated with both global and local attributes, and two real-world datasets. Performance is assessed across five metrics measuring both editability and preservability. Lastly, we conduct a sensitivity analysis on three key components: the balancing weight in the loss function controlled by γ, the number of resolutions k, and the choice of pretrained text embedding model.

4.1 Experimental Setup

Datasets. To evaluate InstructTime, we introduce one new synthetic dataset and use two real-world datasets, one from prior time series editing papers, and one new task. For the **Synthetic** dataset, we generate 18,000 time series samples of length 200, each being a mix of five multi-resolution attributes: trend direction (3 levels: upward, downward, none), trend type (2 levels: linear, quadratic), seasonality (2 levels: present or absent), abrupt mean shift (3 levels: upward, downward, none), and noise level (2 levels: high or low). Trend, seasonality, and noise level represent global patterns, while abrupt mean shifts reflect local patterns. We next use **Air Quality**, which combines two public datasets [10, 17] used in prior works [27, 39], resulting in 3,684 particulate matter (PM2.5) time series from Beijing and London spanning 2013 to 2018. We process the combined datasets following the same procedure as [27]. The resulting time series each have a length of 168 and are influenced by two attributes: location (2 levels) and season (4 levels). Third, we draw from a real neonatal intensive care unit dataset to create a new time series editing dataset called **NICU Heart Rate**, which contains 36,679 heart rate time series of length 300 (10-minute records sampled every 2 seconds) from 2,780 NICU infants between 2012 and 2016 [16, 53]. Each time series is labeled with two clinically meaningful attributes: heart rate variability (2 levels: high or low), and bradycardia events (2 levels: whether or not any heart rate drops below 100 bpm) [1, 41]. To prevent large-scale values during training, both the Air Quality and NICU Heart Rate datasets are normalized by subtracting the global mean and dividing by the global standard deviation, calculated from all values across all time series. All datasets are randomly split into 70% for training, 20% for validation, and 10% for held-out evaluation. Details on datasets can be found in Appendix A.

Compared methods. We compare InstructTime with two state-of-the-art methods—TimeWeaver [39] and TEdit [27]—both of which are implemented with the sampling-based editing procedure introduced in [27]. Since we relax the input requirement from requiring a triplet—comprising a time series, its original attributes, and target attributes—to only taking a time series and target attributes, we adopt the denoising diffusion probabilistic model sampler implemented in [27], which does **not** require the original attributes as input. To use the same computing resource, modifications on configuration are verified to achieve same performance to the original configuration in the attribute-based editing setting.

Implementation details. For InstructTime, the time series encoder uses $k = 8$ parallel CNNs with kernel sizes proportional to the time series length T, using fractions 1, $\frac{2}{3}$, $\frac{1}{2}$, $\frac{1}{3}$, $\frac{1}{4}$, $\frac{1}{6}$, $\frac{1}{8}$, $\frac{1}{10}$. Larger fractions (1, $\frac{2}{3}$, $\frac{1}{2}$) are used to capture global properties such as trend and seasonality, while smaller fractions are intended to encode localized patterns such as abrupt shifts in mean and local variability. For the synthetic data where $T = 200$, such design results in kernel sizes of $\{200, 133, 100, 66, 50, 33, 25, 20\}$ after truncation toward zero. For the air quality data where $T = 168$, the resulting kernel sizes are $\{168, 112, 84, 56, 42, 28, 21, 16\}$. For the NICU data where $T = 300$, the resulting kernel sizes are $\{300, 200, 150, 100, 75, 50, 37, 30\}$. Text instructions are encoded using the pretrained SentenceTransformer model *paraphrase-mpnet-base-v2* [45], which provides effective embedding of rich semantic meaning from multiple sentences and is efficient to compute. The resulting vectors are then processed by $k = 8$ parallel MLPs to generate instruction embeddings $\mathbf{z}_c$. The embeddings $\mathbf{z}_x$ and $\mathbf{z}_c$ have dimension $D = 768$. When training InstructTime, we jointly optimize a contrastive loss and a reconstruction loss, with their relative contributions to the total loss controlled by a balancing weight α. In our experiments, α is set so that the unit contribution of the reconstruction loss is $10^{-\gamma}$ relative to the contrastive loss, with γ set to 1 by default.

Training details. InstructTime is trained on all datasets using a single NVIDIA V100 GPU, requiring 10 GB, 5 GB, and 20 GB of GPU memory for the synthetic, air quality, and NICU datasets, respectively. The corresponding training times are within 0.5, 0.5, and 1 hour. We use an initial learning rate of 1×10^{-4} with a Reduce-on-Plateau learning rate scheduler [37] that decreases the learning rate by a factor of 0.9 when the validation loss plateaus. The scheduler has a patience of 50 epochs and a cool-down period of 20 epochs after each reduction. Optimization is performed using AdamW optimizer [37] with a weight decay of 1×10^{-4}. Early stopping is applied based on validation loss.

Instruction-based vs. attribute-based editing. We compare methods in both instruction-based and attribute-based editing settings. In the **instruction-based** setting, each level of each time series attribute is described using a unique natural language template (e.g., "This is air quality in [Beijing / London]"). The full set of templates is provided in Appendix C. Then sentences of all attributes are concatenated into a paragraph-length instruction. To evaluate generalizability to **unseen expressions**, the description of each attribute level is augmented into 50 different expressions using GPT-4o[25], 30% of which are left out for evaluation. Details on the prompting are provided in Appendix C. Existing methods encode a vector of categorical or continuous attributes into an attribute embedding and process it through MLP layers in the conditional diffusion model. We replace the attribute embedding with the instruction embedding from the pretrained model, ensuring identical input to the MLP layers in our instruction encoder. In the **attribute-based** setting, we input the vector of categorical attributes to InstructTime's instruction encoder, matching the input format used by existing methods.

Evaluation metrics. An editor should demonstrate both editability and preservability [27]. This means identifying the set of K attributes ($\tilde{\mathbf{c}} = \{\tilde{c}_i\}_{i=1}^{K}$) described in an instruction, and then generating a time series $\hat{x}$ where the subset of attributes to edit ($\tilde{\mathbf{c}}_{edit}$) is modified and the subset to preserve ($\tilde{\mathbf{c}}_{prsv}$) is retained. We adopt two established metrics to evaluate the editability and preservability: Log Ratio of Target-to-Source (RaTS) [27] and Dynamic Time Warping distance (DTW) [47]. (1) RaTS measures the extent to which an output time series $\hat{x}$ is more likely to exhibit an attribute $\tilde{c}$ than the input time series x:

$$\text{RaTS}(\hat{x}, x, \tilde{c}) = \log\left(\frac{p(\tilde{c} \mid \hat{x})}{p(\tilde{c} \mid x)}\right),$$

where $p(\tilde{c}|x)$ is the probability of $\hat{x}$ being classified to the correct level of attribute $\tilde{c}$. For each attribute, we compute $p(\tilde{c}|x)$ using a multi-class classifier that appends a softmax prediction head to the architecture of time series encoder $\mathcal{E}_\phi$, trained end-to-end on the held-out dataset. Higher $\text{RaTS}(\hat{x}, x, \tilde{c}_{\text{edit}})$ indicates better editability. The absolute value $|\text{RaTS}(\hat{x}, x, \tilde{c}_{\text{prsv}})|$ measures how well the output $\hat{x}$ preserves the remaining attributes $\tilde{c}_{\text{prsv}}$ of the input x; values closer to zero indicate better preservability [27]. Note that we use an end-to-end trained classifier to calculate $p(\tilde{c}|x)$ for each attribute, rather than using the retrieval softmax probability computed from the shared embedding space as done by [27], because our generation process used the shared embedding space, and using retrieval softmax probability may favor our edits. Our results remain consistent across runs, while uncertainty in metrics can occur due to the sampling procedure in prior methods.

(2) DTW measures the similarity between two time series by finding an optimal alignment that minimizes the cumulative point-wise difference [47]. We use ΔDTW, defined below, to measure how much closer $\hat{x}$ is to the data observed with target attributes $\tilde{\mathbf{c}}$ (denoted as $\mathcal{D}_{\tilde{\mathbf{c}}}$) compared to x:

$$\Delta\text{DTW}(\hat{x}, x, \mathcal{D}_{\tilde{\mathbf{c}}}) = \text{median}_{x_{\tilde{\mathbf{c}}} \in \mathcal{D}_{\tilde{\mathbf{c}}}} \left[\text{DTW}(x, x_{\tilde{\mathbf{c}}}) - \text{DTW}(\hat{x}, x_{\tilde{\mathbf{c}}})\right].$$

A lower negative ΔDTW suggests better editability because the edited time series is closer to the real data with the target attributes. Following [27], for synthetic data with ground truth, we also report MSE and MAE to evaluate the point-wise closeness between the ground truth x_{gt} and the generated time series $\hat{x}$.

4.2 Editing Quality

Results for our main comparisons can be seen in Table 2, where we compare each method's editability and preservability for both instruction-based and attribute-based time series editing. First, we observe that InstructTime is the state-of-the-art editor in the instruction-based setting across all datasets. Under the instruction-based setting, InstructTime achieves higher RaTS, MSE, and MAE, along with lower |RaTS| on the synthetic dataset, compared to alternative editors. The higher RaTS indicates that the edited time series more strongly exhibits the target conditions to edit than the input, while the lower |RaTS| reflects better preservation of the conditions that are not meant to change. The lower ΔDTW further shows that the edited time series is closer to the observed ones with the target conditions. The lower MSE and MAE indicate more accurate point-wise alignment between the edits and the synthetic ground truth. These results suggest that InstructTime more effectively decomposes multiple conditions from an instruction—both those that match the input and those that differ—and edits the time series toward the target conditions while preserving

		Synthetic					Air quality			NICU heart rate		
		ΔDTW ↓	RaTS ↑	|RaTS| ↓	MSE ↓	MAE ↓	ΔDTW ↓	RaTS ↑	|RaTS| ↓	ΔDTW ↓	RaTS ↑	|RaTS| ↓
Instruction-based	Time Weaver	-11.05	5.93	0.63	3.50	1.54	-0.97	0.17	0.66	-0.15	0.13	1.03
	TEdit	-9.66	5.99	0.86	3.45	1.52	-0.90	0.12	0.65	-2.41	0.15	1.03
	InstructTime	**-14.62**	**6.87**	**0.02**	**2.26**	**1.18**	**-1.84**	**0.51**	**0.62**	**-4.38**	**0.18**	**0.42**
Attribute-based	Time Weaver	-12.19	6.65	0.15	3.58	1.54	-1.26	0.31	0.73	-1.55	0.15	1.40
	TEdit	-10.66	**6.98**	**0.08**	3.96	1.61	**-1.71**	0.30	0.72	-1.96	0.18	**1.38**
	InstructTime	**-13.62**	6.77	0.20	**2.38**	**1.25**	-1.62	**0.43**	**0.70**	**-4.94**	**0.26**	1.41

Table 2: Comparison of model editing quality across datasets under instruction-based vs. attribute-based editing settings. Scores are averaged across attributes. InstructTime results are reported at $w = 0.9$.

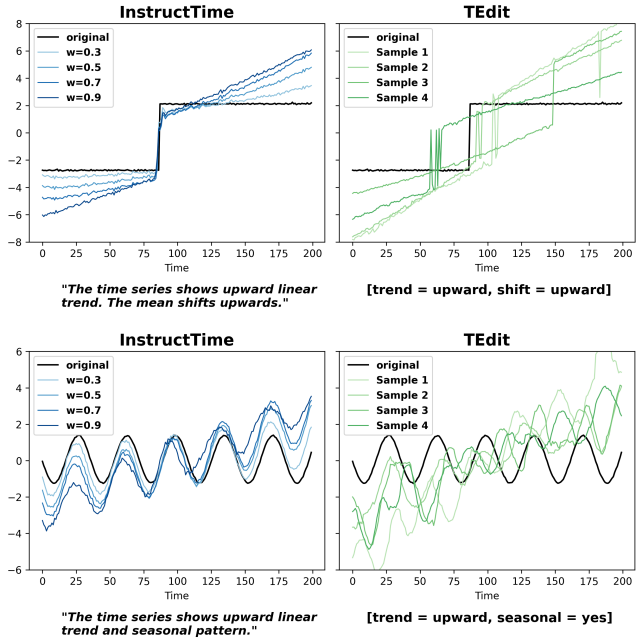


Figure 3: Interpolated editing. As the editing strength w increases, InstructTime gradually edits the flat trend into an upward trend while preserving either the local pattern of upward shifts in the mean (upper) or the global pattern of seasonality (lower). Diffusion-based TEdit generates upward trends while preserving other attributes, but with varying slopes and scattered distances from the input.

the unique structure of the input instance. We next note that InstructTime's time series encoder and decoder are agnostic to the natural language instruction encoder. Therefore, we also consider the attribute-based setting, which involves no instructions. We first note that InstructTime is highly-competitive, even outperforming the comparisons in multiple metrics and datasets. This suggests that our architecture is generalizable and likely drives some of the performance in the instruction-based setting. We also observe that TEdit shows strong editability and preservability, while InstructTime achieves lower MSE and MAE on synthetic data, indicating better point-wise alignment. These results suggest that diffusion-based methods can faithfully edit target attributes, but the output time series are scattered more widely from the input time series due to sampling randomness. Editing quality on individual attributes can be found in Appendix D, alongside edited examples.

4.3 Interpolated Editing

As the editing strength w increases, the edited time series should progressively reflect the target edited attributes $\tilde{\mathbf{c}}_{\text{edit}}$ to a greater extent, while maintaining high preservability of $\tilde{\mathbf{c}}_{\text{prsv}}$. Figure 3 shows a case study comparing the editing procedures by InstructTime and TEdit. As editing strength w increases, InstructTime gradually turns a flat trend into upward trends with increasing slope, while preserving local shifts or global seasonality. In contrast, TEdit produces upward trends with random slopes and scattered distances from the original while preserving other attributes. Figure 4 shows the progressive change in editability (ΔDTW, RaTS) and preservability (|RaTS|) by InstructTime as the editing strength w increases, compared to the fixed performances by TEdit and Time Weaver. As w increases, the target attributes become more prominent in the edited time series, resulting in stronger editability. Larger w also leads to greater divergence from the input time series and lower preservability, as changes to target attributes may introduce turbulence in other attributes meant to be preserved. Despite this inherent trade-off, InstructTime maintains comparable preservability as w approaches 1 and achieves better preservability in the low-w regime than other methods. Compared to attribute-based editing, InstructTime maintains high editing quality in the instruction-based setting, while other methods show reduced editability and preservability, as indicated by higher ΔDTW, lower RaTS, and higher |RaTS|.

4.4 Generalizability to Unseen Instructions

An advantage of editing time series using natural language instructions is the potential to generalize to rich semantics of time series conditions, including (1) new expressions describing known conditions and (2) instructions on entirely unseen conditions.

For the first case, leveraging a pretrained text encoder allows the instruction encoder to map new expressions of known conditions close to seen ones in the embedding space due to their high cosine similarity. As a result, InstructTime is expected to generalize to such expressions (augmented by GPT-4o) in a zero-shot manner. As shown in Table 3, InstructTime achieves comparable editability and preservability when instructions use unseen expressions to describe time series attributes seen during training, and continues to outperform existing methods.

For the second case, we assess InstructTime's leave-one-out generalizability on the 3-level attributes of the synthetic time series: *trend direction* and *shift in mean*. For each attribute, one level (e.g., "upward trend") is excluded during training. This setup simulates a realistic scenario where the model encounters new but semantically

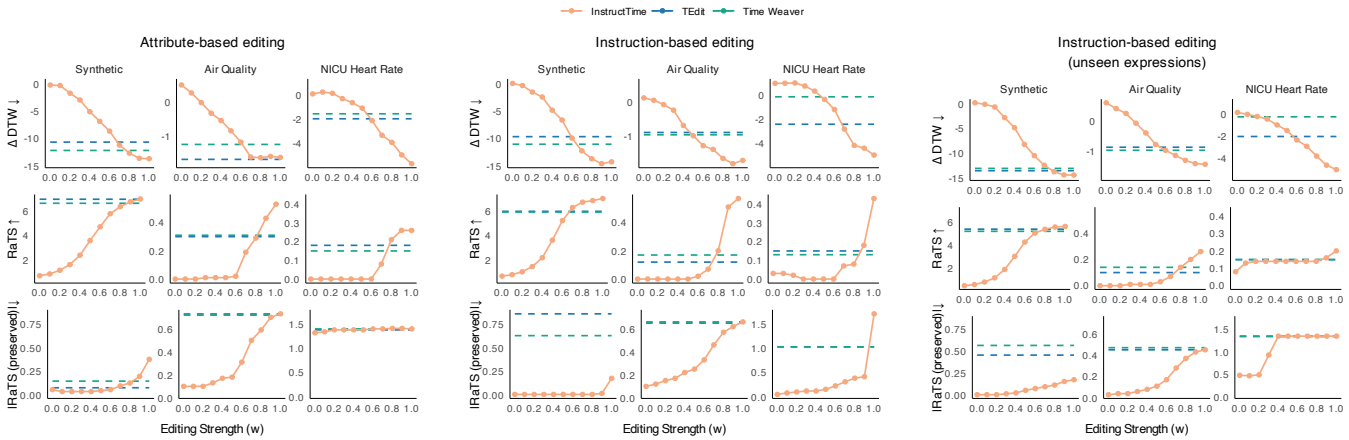


Figure 4: Progressive change in editability and preservability by InstructTime as editing strength w increases from 0 to 1 (step 0.1). TEdit and Time Weaver edit time series without controllable strength.

		Synthetic					Air quality			NICU heart rate		
		ΔDTW ↓	RaTS ↑	|RaTS| ↓	MSE ↓	MAE ↓	ΔDTW ↓	RaTS ↑	|RaTS| ↓	ΔDTW ↓	RaTS ↑	|RaTS| ↓
Instruction-based (unseen-phrasing)	Time Weaver	-12.95	5.24	0.57	3.25	1.48	-0.97	0.14	0.47	-0.27	0.15	**1.34**
	TEdit	-13.41	5.43	0.46	3.65	1.56	-0.87	0.10	0.45	-2.04	0.15	1.35
	InstructTime	**-14.26**	**5.62**	**0.16**	**2.96**	**1.39**	**-1.41**	**0.20**	**0.43**	**-4.62**	**0.16**	1.35

Table 3: Editing quality with unseen expressions in instruction-based editing. Scores are averaged across attributes. InstructTime scores are computed at $w = 0.9$.

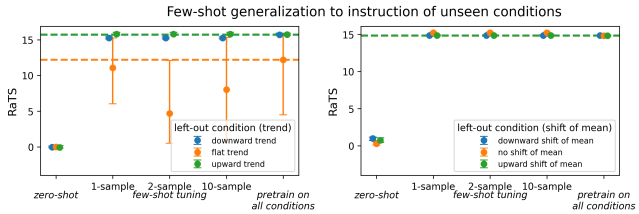


Figure 5: Few-shot tuning InstructTime on unseen conditions approaches the editability by model trained on all levels.

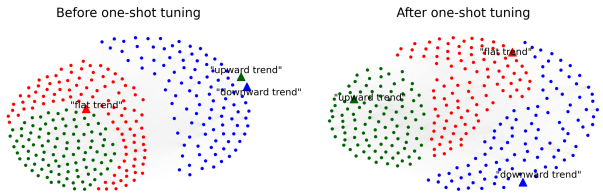


Figure 6: Embedding space before and after tuning with one example from the unseen condition (upward trend). Triangles and circles represent text and time series embeddings, respectively; colors indicate three trend levels. Embeddings are pairwise connected by cosine similarity and visualized in a force-directed layout.

related conditions at inference time. For example, a flat trend can be seen as an average state between upward and downward trends. We then apply the few-shot tuning procedure using 1, 2, or 10 example instruction–time series pairs from the left-out level. The goal is to test whether limited supervision enables the model to generate edits by leveraging the semantic relationship between unseen and seen conditions. As shown in Figure 5, the model trained on two levels cannot directly generalize to the left-out level in a zero-shot manner, as indicated by the near-zero RaTS scores. However, tuning with a few examples—even just one—results in editability on the unseen level comparable to the baseline where the model is trained on all three levels.

Figure 6 visualizes the re-calibration of time series and instruction embeddings in the shared embedding space after tuning with a single example (one-shot). Before tuning (left panel), the model is trained on two trend levels: flat (red) and downward (blue). Dots represent embeddings of 100 time series per level; triangles denote their instruction embeddings. The instruction and time series embeddings align well for the seen levels, but for the unseen "upward trend" (green), the triangle and dots are far apart in the embedding space. The model is then tuned on a small set of instruction–time series pairs, generated by providing an example pair from the unseen level "upward trend" and editing it toward the seen levels to create a few synthetic samples for tuning. This allows the model to adjust its internal representation of the unseen condition in terms of its relative position and distance to the seen conditions in the embedding space. As shown in the right panel, the time series and instruction embeddings are well calibrated across all three levels, highlighting InstructTime's ability to generalize under limited supervision.

4.5 Ablation Studies

4.5.1 Balancing weight. When training InstructTime, we jointly optimize a contrastive loss and a reconstruction loss, with their relative contributions to the total loss controlled by a balancing weight. The weight is set so that the unit contribution of the reconstruction loss to the total loss is $10^{-\gamma}$ relative to the contrastive loss. A larger γ prompts the model to prioritize the alignment between

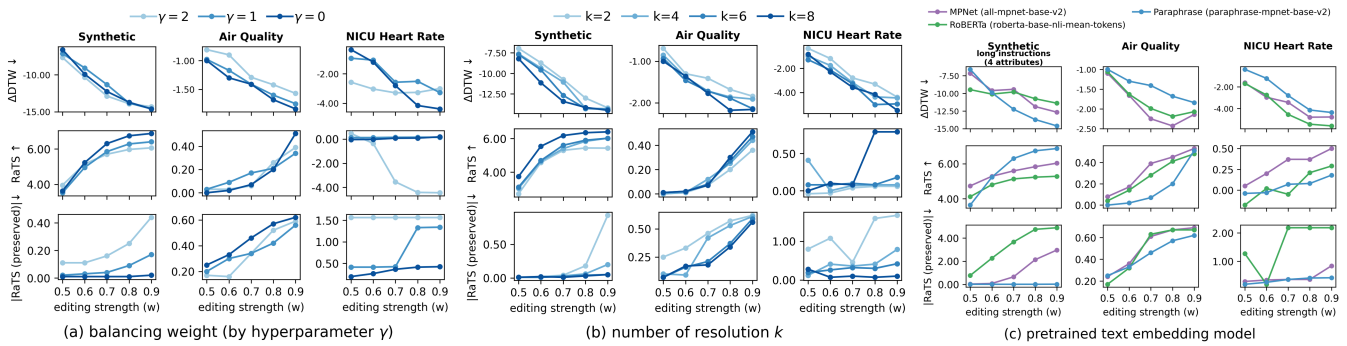


Figure 7: Sensitivity analysis on key components of InstructTime in instruction-based editing.

time series and instruction embeddings, compared to reconstruction. We experiment with $\gamma \in \{0, 1, 2\}$ across all three datasets, corresponding to unit reconstruction-to-contrastive loss ratios of $\{1, 0.1, 0.01\}$, respectively. As shown in Figure 7 (a), both $\gamma = 0$ and $\gamma = 1$ yield similar optimal editability (ΔDTW, RaTS) and preservability (|RaTS|) across datasets. For the synthetic and NICU heart rate datasets, however, setting $\gamma = 2$ overly de-prioritizes reconstruction, resulting in degraded RaTS and |RaTS|. These suggest while the optimal γ may vary by dataset, setting $\gamma \in \{\mathbf{0}, \mathbf{1}\}$ provides a robust and effective default.

4.5.2 Resolution k. The multi-resolution time series encoder consists of k CNNs, each with a kernel size set to a different fraction of the input time series length, allowing the model to capture patterns at multiple resolutions. We evaluate the effect of varying k on model performance, with $k = 2, 4, 6, 8$. As shown in Figure 7 (b), $\mathbf{k} \in \{\mathbf{6}, \mathbf{8}\}$ generally yield better editability (ΔDTW, RaTS) and preservability (|RaTS|) across datasets, whereas $k = 2$ is overly simplistic for capturing complex patterns at varying resolutions for the synthetic and NICU datasets.

4.5.3 Pretrained text embedding model. The instruction encoder leverages a frozen pretrained text embedding model to firstly convert instructions into numerical vectors. The choice of pretrained model should be sensitive to multiple attributes described in a short-paragraph (multi-sentence) instruction to ensure that both the conditions to edit and to preserve are accurately reflected in the edited time series. We next experiment with two additional publicly available, computationally efficient pretrained models—MPNet (*all-mpnet-base-v2*) [51] and RoBERTa (*roberta-base-nli-mean-tokens*) [35]—in comparison with our default choice, Paraphrase MPNet (*paraphrase-mpnet-base-v2*) [45]. As shown in Figure 7(c), all models show improved editability with increasing editing strength. RoBERTa yields the lowest preservability. MPNet performs better on short instructions, while Paraphrase MPNet notably outperforms other models on synthetic data with long instructions describing 4 attributes. This result suggests that MPNet models are effective candidates, as models fine-tuned for paraphrase tasks perform better on long instructions.

5 Conclusion

In this paper, we introduce Instruction-based Time Series Editing, a novel task that enables fine-grained modification of time series based on natural language instructions. Unlike prior attribute-based approaches, our formulation allows flexible conditioning on free-form text, aligning more closely with real-world scenarios where text captures nuanced, individualized context. We propose InstructTime, the first instruction-based time series editor that uses contrastive learning to embed time series and instructions in a shared space, and decodes interpolated embeddings to generate edits under varying degrees of condition influence. Instruction-based time series editing raises broader considerations, which can drive impactful future research. First, foundation models can generate time series using language prompts, but they are not suited for time series editing yet, which requires context-dependent, data-driven models that allow high-fidelity, in-context edits. Second, fine-grained control over editing strength is essential for instruction-based editors due to the discrete nature of language. Interpolated editing also supports hypothesis generation on how time series transition under varying conditions, but remains exploratory. Other future directions may consider: extending the architecture to a variational form [13] to enable sampling-based editing with controllable strength and reduced global averaging; enabling step-wise time series editing guided by natural language instructions describing time-dependent conditions; and evaluating real-world utility in domains such as healthcare where time series are accompanied by nuanced and individualized text such as nurse documentation or patient narrative.

6 Acknowledgements

We thank University of Virginia's High Performance Computing team for computing resources. We thank the reviewers for their constructive feedback, which helps improve the manuscript in many important ways. Thomas Hartvigsen was supported in part by a CapitalOne faculty fellowship and the University of Virginia's National Security Data and Policy Institute through the U.S. Department of Defense Contracting Activity, Award Number #2024-24070100001. Contact author: Jiaxing Qiu (jq2uw@virginia.edu).

References

[1] Namasivayam Ambalavanan, Debra E Weese-Mayer, Anna Maria Hibbs, Nelson Claure, John L Carroll, J Randall Moorman, Eduardo Bancalari, Aaron Hamvas, Richard J Martin, Juliann M Di Fiore, et al. 2023. Cardiorespiratory monitoring data to predict respiratory outcomes in extremely preterm infants. *American journal of respiratory and critical care medicine* 208, 1 (2023), 79–97.

[2] Gaby Baasch, Guillaume Rousseau, and Ralph Evins. 2021. A Conditional Generative adversarial Network for energy use in multiple buildings using scarce data. *Energy and AI* 5 (2021), 100087. doi:10.1016/j.egyai.2021.100087

[3] Kasun Bandara, Hansika Hewamalage, Yuan-Hao Liu, Yanfei Kang, and Christoph Bergmeir. 2021. Improving the accuracy of global forecasting models using time series data augmentation. *Pattern Recognition* 120 (2021), 108148.

[4] Samuel Bowman, Luke Vilnis, Oriol Vinyals, Andrew Dai, Rafal Jozefowicz, and Samy Bengio. 2016. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL conference on computational natural language learning*. 10–21.

[5] Charlotte Bunne, Yusuf Roohani, Yanay Rosen, Ankit Gupta, Xikun Zhang, Marcel Roed, Theo Alexandrov, Mohammed AlQuraishi, Patricia Brennan, Daniel B Burkhardt, et al. 2024. How to build the virtual cell with artificial intelligence: Priorities and opportunities. *Cell* 187, 25 (2024), 7045–7063.

[6] Huili Cai, Xiang Zhang, and Xiaofeng Liu. 2025. Semi-Supervised End-To-End Contrastive Learning For Time Series Classification. arXiv:2310.08848 [cs.LG] https://arxiv.org/abs/2310.08848

[7] Defu Cao, Wen Ye, Yizhou Zhang, and Yan Liu. 2024. TimeDiT: General-purpose Diffusion Transformers for Time Series Foundation Model. (Sept. 2024). arXiv:2409.02322 [cs.LG] doi:10.48550/ARXIV.2409.02322

[8] Jialin Chen, Ziyu Zhao, Gaukhar Nurbek, Aosong Feng, Ali Maatouk, Leandros Tassiulas, Yifeng Gao, and Rex Ying. 2025. TRACE: Grounding Time Series in Context for Multimodal Embedding and Retrieval. arXiv:2506.09114 [cs.LG] https://arxiv.org/abs/2506.09114

[9] Muxi Chen, Zhijian Xu, Ailing Zeng, and Qiang Xu. 2023. FrAug: Frequency domain augmentation for time series forecasting. *arXiv preprint arXiv:2302.09292* (2023).

[10] Song Chen. 2017. *Beijing Multi-Site Air Quality*. doi:10.24432/C5RK5G Dataset.

[11] Razvan-Gabriel Cirstea, Bin Yang, Chenjuan Guo, Tung Kieu, and Shirui Pan. 2022. Towards spatio-temporal aware traffic time series forecasting. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2900–2913.

[12] Andrea Coletta, Sriram Gopalakrishnan, Daniel Borrajo, and Svitlana Vyetrenko. 2023. On the constrained time-series generation problem. *Advances in Neural Information Processing Systems* 36 (2023), 61048–61059.

[13] Tim R Davidson, Luca Falorsi, Nicola De Cao, Thomas Kipf, and Jakub M Tomczak. 2018. Hyperspherical variational auto-encoders. *arXiv preprint arXiv:1804.00891* (2018).

[14] Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. 2021. TimeVAE: A Variational Auto-Encoder for Multivariate Time Series Generation. (Nov. 2021). arXiv:2111.08095 [cs.LG] doi:10.48550/ARXIV.2111.08095

[15] Cristóbal Esteban, Stephanie L Hyland, and Gunnar Rätsch. 2017. Real-valued (medical) time series generation with recurrent conditional gans. *arXiv preprint arXiv:1706.02633* (2017).

[16] Ian German Mesner. 2024. Pediatric Academic Societies 2024 NICU Mortality Prediction Challenge. doi:10.18130/V3/5UYB4U

[17] Ruvinda Godahewa, Christoph Bergmeir, Geoffrey Webb, Rob Hyndman, and Pablo Montero-Manso. 2020. *KDD Cup Dataset (without Missing Values) (Version 4)*. doi:10.5281/zenodo.4656756 Data set.

[18] Alexandru Grigoraș and Florin Leon. 2024. Synthetic time series generation for decision intelligence using large language models. *Mathematics* 12, 16 (2024), 2494.

[19] Mohamed Hamdouche, Pierre Henry-Labordere, and Huyên Pham. 2023. Generative modeling for time series via Schrödinger bridge. (April 2023). arXiv:2304.05093 [math.OC] doi:10.48550/ARXIV.2304.05093

[20] Shilei Hao, Zhihai Wang, Afanasiev D. Alexander, Jidong Yuan, and Wei Zhang. 2023. MICOS: Mixed supervised contrastive learning for multivariate time series classification. *Knowledge-Based Systems* 260 (2023), 110158. doi:10.1016/j.knosys.2022.110158

[21] Tong He, Weilin Huang, Yu Qiao, and Jian Yao. 2016. Accurate text localization in natural image with cascaded convolutional text network. *arXiv preprint arXiv:1603.09423* (2016).

[22] Genc Hoxha, Olivér Angyal, and Begüm Demir. 2025. Self-Supervised Cross-Modal Text-Image Time Series Retrieval in Remote Sensing. arXiv:2501.19043 [cs.CV] https://arxiv.org/abs/2501.19043

[23] Jie Hu, Zhanao Hu, Tianrui Li, and Shengdong Du. 2023. A contrastive learning based universal representation for time series forecasting. *Information Sciences* 635 (2023), 86–98. doi:10.1016/j.ins.2023.03.143

[24] Yu-Hao Huang, Chang Xu, Yueying Wu, Wu-Jun Li, and Jiang Bian. 2025. Timedp: Learning to generate multi-domain time series with domain prompts. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 17520–17527.

[25] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).

[26] Yushan Jiang, Wenchao Yu, Geon Lee, Dongjin Song, Kijung Shin, Wei Cheng, Yanchi Liu, and Haifeng Chen. 2025. Explainable multi-modal time series prediction with LLM-in-the-Loop. *arXiv preprint arXiv:2503.01013* (2025).

[27] Baoyu Jing, Shuqi Gu, Tianyu Chen, Zhiyu Yang, Dongsheng Li, Jingrui He, and Kan Ren. 2024. Towards editing time series. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[28] Delanyo Kwame Bensah Kulevome, Hong Wang, Bernard Mawuli Cobbinah, Ernest Smith Mawuli, and Rajesh Kumar. 2024. Effective time-series data augmentation with analytic wavelets for bearing fault diagnosis. *Expert Systems with Applications* 249 (2024), 123536.

[29] Su-In Lee and Eric J Topol. 2024. The clinical potential of counterfactual AI models. *The Lancet* 403, 10428 (2024), 717.

[30] Hao Li, Yuhao Huang, Chang Xu, Viktor Schlegel, Renhe Jiang, Riza Batista-Navarro, Goran Nenadic, and Jiang Bian. 2025. Bridge: Bootstrapping text to control time-series generation via multi-agent iterative optimization and diffusion modelling. *arXiv preprint arXiv:2503.02445* (2025).

[31] Michelle M Li, Kevin Li, Yasha Ektefaie, Shvat Messica, and Marinka Zitnik. 2025. Controllable Sequence Editing for Counterfactual Generation. *arXiv preprint arXiv:2502.03569* (2025).

[32] Yuqi Lin, Minghao Chen, Wenxiao Wang, Boxi Wu, Ke Li, Binbin Lin, Haifeng Liu, and Xiaofei He. 2023. Clip is also an efficient segmenter: A text-driven approach for weakly supervised semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 15305–15314.

[33] Jiexi Liu and Songcan Chen. 2023. TimesURL: Self-supervised Contrastive Learning for Universal Time Series Representation Learning. arXiv:2312.15709 [cs.LG] https://arxiv.org/abs/2312.15709

[34] Xinhe Liu and Wenmin Wang. 2024. Deep Time Series Forecasting Models: A Comprehensive Survey. *Mathematics* 12, 10 (2024). doi:10.3390/math12101504

[35] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[36] Ziyu Liu, Azadeh Alavi, Minyi Li, and Xiang Zhang. 2023. Self-Supervised Contrastive Learning for Medical Time Series: A Systematic Review. *Sensors* 23, 9 (2023). doi:10.3390/s23094221

[37] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).

[38] Navid Mohammadi Foumani, Lynn Miller, Chang Wei Tan, Geoffrey I Webb, Germain Forestier, and Mahsa Salehi. 2024. Deep learning for time series classification and extrinsic regression: A current survey. *Comput. Surveys* 56, 9 (2024), 1–45.

[39] Sai Shankar Narasimhan, Shubhankar Agarwal, Oguzhan Akcin, Sujay Sanghavi, and Sandeep Chinchali. 2024. Time weaver: A conditional time series generation model. *arXiv preprint arXiv:2403.02682* (2024).

[40] Bin Qian, Yong Xiao, Zhenjing Zheng, Mi Zhou, Wanqing Zhuang, Sen Li, and Qianli Ma. 2020. Dynamic multi-scale convolutional neural network for time series classification. *IEEE access* 8 (2020), 109732–109746.

[41] Jiaxing Qiu, Juliann M Di Fiore, Narayanan Krishnamurthi, Premananda Indic, John L Carroll, Nelson Claure, James S Kemp, Phyllis A Dennery, Namasivayam Ambalavanan, Debra E Weese-Mayer, et al. 2024. Highly comparative time series analysis of oxygen saturation and heart rate to predict respiratory outcomes in extremely preterm infants. *Physiological measurement* 45, 5 (2024), 055025.

[42] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PmLR, 8748–8763.

[43] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125* 1, 2 (2022), 3.

[44] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. 2021. Autoregressive Denoising Diffusion Models for Multivariate Probabilistic Time Series Forecasting. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 8857–8868. https://proceedings.mlr.press/v139/rasul21a.html

[45] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. http://arxiv.org/abs/1908.10084

[46] Lei Ren, Haiteng Wang, and Yuanjun Laili. 2024. Diff-MTS: Temporal-Augmented Conditional Diffusion-Based AIGC for Industrial Time Series Toward the Large Model Era. *IEEE Transactions on Cybernetics* 54, 12 (2024), 7187–7197. doi:10.1109/TCYB.2024.3462500

[47] Hiroaki Sakoe and Seibi Chiba. 2003. Dynamic programming algorithm optimization for spoken word recognition. *IEEE transactions on acoustics, speech, and signal processing* 26, 1 (2003), 43–49.

[48] Artemios-Anargyros Semenoglou, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2023. Data augmentation for univariate time series forecasting with neural networks. *Pattern Recognition* 134 (2023), 109132.

[49] Alexander Sommers, Logan Cummins, Sudip Mittal, Shahram Rahimi, Maria Seale, Joseph Jaboure, and Thomas Arnold. 2024. A Survey of Transformer Enabled Time Series Synthesis. (June 2024). arXiv:2406.02322 [cs.LG] doi:10.48550/ARXIV.2406.02322

[50] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2020. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502* (2020).

[51] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. Mpnet: Masked and permuted pre-training for language understanding. *Advances in neural information processing systems* 33 (2020), 16857–16867.

[52] Michael Studdert-Kennedy. 2005. How did language go discrete. *Language origins: Perspectives on evolution, ed. M. Tallerman* (2005), 48–67.

[53] Brynne A Sullivan, Alvaro G Moreira, Ryan M McAdams, Lindsey A Knake, Ameena Husain, Jiaxing Qiu, Avinash Mudireddy, Abrar Majeedi, Wissam Shalish, Douglas E Lake, et al. 2024. Comparing machine learning techniques for neonatal mortality prediction: insights from a modeling competition. *Pediatric research* (2024), 1–7.
[54] Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. 2021. CSDI: Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 24804–24816. https://proceedings.neurips.cc/paper_files/paper/2021/file/cfe8504bda37b575c70ee1a8276f3486-Paper.pdf
[55] Guy Tevet, Brian Gordon, Amir Hertz, Amit H Bermano, and Daniel Cohen-Or. 2022. Motionclip: Exposing human motion generation to clip space. In *European Conference on Computer Vision*. Springer, 358–374.
[56] Muhang Tian, Bernie Chen, Allan Guo, Shiyi Jiang, and Anru R Zhang. 2024. Reliable generation of privacy-preserving synthetic electronic health record time series via diffusion models. *Journal of the American Medical Informatics Association* 31, 11 (09 2024), 2529–2539. arXiv:https://academic.oup.com/jamia/article-pdf/31/11/2529/59813606/ocae229.pdf doi:10.1093/jamia/ocae229
[57] José F. Torres, Dalil Hadjout, Abderrazak Sebaa, Francisco Martínez-Álvarez, and Alicia Troncoso. 2021. Deep Learning for Time Series Forecasting: A Survey. *Big Data* 9, 1 (Feb. 2021), 3–21. doi:10.1089/big.2020.0159
[58] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[59] Claudio Meneses Villegas, Jorge Littin Curinao, David Coo Aqueveque, Juan Guerrero-Henríquez, and Martín Vargas Matamala. 2024. Data augmentation and hierarchical classification to support the diagnosis of neuropathies based on time series analysis. *Biomedical Signal Processing and Control* 95 (2024), 106302.
[60] Clinton Wang and Polina Golland. 2023. Interpolating between images with diffusion models. (2023).
[61] Sheng Wang, Xi Chen, Dongliang Ma, Chen Wang, Yong Wang, Honggang Qi, Gongjian Zhou, Qingli Li, and Min Liu. 2023. MIANet: Multi-level temporal information aggregation in mixed-periodicity time series forecasting tasks. *Engineering Applications of Artificial Intelligence* 121 (2023), 106175.
[62] Haomin Wen, Youfang Lin, Yutong Xia, Huaiyu Wan, Qingsong Wen, Roger Zimmermann, and Yuxuan Liang. 2023. Diffstg: Probabilistic spatio-temporal graph forecasting with denoising diffusion models. In *Proceedings of the 31st ACM International Conference on Advances in Geographic Information Systems*. 1–12.
[63] Qingsong Wen, Liang Sun, Fan Yang, Xiaomin Song, Jingkun Gao, Xue Wang, and Huan Xu. 2020. Time series data augmentation for deep learning: A survey. *arXiv preprint arXiv:2002.12478* (2020).
[64] Gerald Woo, Chenghao Liu, Doyen Sahoo, Akshat Kumar, and Steven Hoi. 2022. CoST: Contrastive Learning of Disentangled Seasonal-Trend Representations for Time Series Forecasting. arXiv:2202.01575 [cs.LG] https://arxiv.org/abs/2202.01575
[65] Jianhua Yang, Yan Huang, Zhanyu Ma, and Liang Wang. 2021. CMF: Cascaded multi-model fusion for referring image segmentation. In *2021 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2289–2293.
[66] Jinsung Yoon, Daniel Jarrett, and Mihaela van der Schaar. 2019. Time-series Generative Adversarial Networks. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/c9efe5f26cd17ba6216bbe2a7d26d490-Paper.pdf
[67] Jinsung Yoon, Daniel Jarrett, and Mihaela Van der Schaar. 2019. Time-series generative adversarial networks. *Advances in neural information processing systems* 32 (2019).
[68] Hao Yu, Chu Xin Cheng, Runlong Yu, Yuyang Ye, Shiwei Tong, Zhaofeng Liu, and Defu Lian. 2025. How to Unlock Time Series Editing? Diffusion-Driven Approach with Multi-Grained Control. *arXiv preprint arXiv:2506.05276* (2025).
[69] Qihang Yu, Ju He, Xueqing Deng, Xiaohui Shen, and Liang-Chieh Chen. 2023. Convolutions die hard: Open-vocabulary segmentation with single frozen convolutional clip. *Advances in Neural Information Processing Systems* 36 (2023), 32215–32234.
[70] Xiaochen Zheng, Xingyu Chen, Manuel Schürch, Amina Mollaysa, Ahmed Allam, and Michael Krauthammer. 2024. Simple Contrastive Representation Learning for Time Series Forecasting. arXiv:2303.18205 [cs.LG] https://arxiv.org/abs/2303.18205
[71] Xu Zheng, Tianchun Wang, Wei Cheng, Aitian Ma, Haifeng Chen, Mo Sha, and Dongsheng Luo. 2024. Parametric Augmentation for Time Series Contrastive Learning. arXiv:2402.10434 [cs.LG] https://arxiv.org/abs/2402.10434

Appendix

A Datasets

A.1 Synthetic Time Series

We synthesize time series of length $T = 200$ by composing multiple attributes. Each series is formed as $\mathbf{x} = \mathbf{x}_{\text{trend}} + \mathbf{x}_{\text{season}} + \mathbf{x}_{\text{shift}} + \mathbf{x}_{\text{noise}}$, where each component is generated according to the attribute configurations below. We consider four multi-resolution attributes: trend type (5 levels: linear upward, linear downward, quadratic upward, quadratic downward, none), seasonality (2 levels: present or absent), abrupt mean shift (3 levels: upward, downward, none), and noise level (2 levels: high or low). For each level of each attribute, we generate 300 time series. Combining components yields $5 \times 2 \times 3 \times 2 = 60$ unique configurations, each with 300 samples, totaling 18,000 time series. Trend and seasonality capture global patterns as functions of time, while abrupt mean shifts model local changes at specific time points. We randomly split the dataset into training, validation, and test sets using a 70–20–10 ratio, resulting in 12,600 training, 3,600 validation, and 1,800 test samples.

Trend. We generate synthetic time series with five trend types: flat, upward-linear, downward-linear, upward-quadratic, and downward-quadratic. Flat sequences are constant-valued, with each point set to $c \sim \mathcal{U}(-5, 5)$. Linear trends follow $x_t = at$, where $a \sim \mathcal{U}(0.05, 0.20)$; a is positive for upward linear and negated for downward linear. Each linear sequence is centered to zero mean and shifted by $m \sim \mathcal{U}(-5, 5)$. Quadratic trends use $x_t = a(t + b)^2$, with curvature $a \sim \mathcal{U}(10^{-4}, 5{\times}10^{-4})$ and shift $b \sim \mathcal{U}(0, 50)$; a is positive for upward-quadratic and negated for downward-quadratic. When needed, sequences are shifted upward to ensure non-negativity, then centered and offset by m. These constructions allow controlled variation in trend direction and curvature.

Seasonality. We generate time series with and without seasonal structure. The *seasonal* group has two equal subsets: one with a *single seasonal component* $x_t = A \sin\left(\frac{2\pi t}{P} + \phi\right)$, and one with *multiple seasonal components*, formed by summing k such terms with varying amplitudes A, periods P, and phases ϕ. All sequences include additive Gaussian noise. This design allows controlled analysis across different levels of seasonal complexity. The *no-seasonality* group uses low-amplitude, long-period oscillations that are effectively indistinguishable from random noise.

Abrupt shift in mean. To model localized changes, we simulate series with either no shift, an upward shift, or a downward shift. A *no-shift* series is centered at zero and perturbed by small Gaussian noise $\epsilon_t \sim \mathcal{N}(0, 0.05^2)$. For shifted series, an abrupt change occurs at a random time $t_s \in [0.1T,\ 0.9T]$, after which all values are offset by $\Delta \sim \mathcal{U}(15, 20)$. We apply $+\Delta$ for *upward* and $-\Delta$ for *downward* shifts, producing persistent regime changes.

Noise (local variability). We simulate time series with either low or high local variability. Each sequence is composed of additive Gaussian noise $\epsilon_t \sim \mathcal{N}(0,\ \sigma^2)$, where the standard deviation σ is drawn from a variability-specific range: $\sigma \sim \mathcal{U}(0.01,\ 0.1)$ for low variability and $\sigma \sim \mathcal{U}(1.5,\ 2.0)$ for high variability.

A.2 Air Quality

We combine two public air quality datasets [10, 17] previously used in [27, 39], resulting in 3684 time series of hourly PM2.5 from Beijing and London spanning 03/01/2013 to 03/31/2018. The first source [17], originally used in the KDD Cup 2018, contains hourly air quality measurements from 35 stations in Beijing and 24 in London between January 2017 and March 2018, measuring air pollutants

Dataset	Attribute	Levels	Templates
Synthetic	Trend	[no trend, upward linear, downward linear, upward quadratic, downward quadratic]	"No trend." "The time series shows [upward linear, downward linear, upward quadratic, downward quadratic] trend."
	Seasonality	[no, yes]	"No seasonal pattern." "The time series exhibits a seasonal pattern."
	Shift in mean	[no shift, upward shift, downward shift]	"No sharp shifts." "The mean of the time series shifts [upwards, downwards]."
	Local variability	[low, high]	"The time series exhibits [low, high] variability."
Air quality	City	[Beijing, London]	"This is air quality in [Beijing, London]."
	Season	[spring, summer, autumn, winter]	"The season is [spring, summer, autumn, winter]."
NICU heart rate	Variability	[low, high]	"The time series exhibits [low, high] variability."
	Bradycardia events	[no, yes]	"[No Bradycardia events. | Bradycardia events happened.]"

Table 4: Natural language templates for each attribute level.

such as PM2.5, PM10, NO_2, and O_3. The second source [10] provides hourly PM2.5 data from 12 stations in Beijing from 2013 to 2017. We process the combined dataset following the procedure in [17]: retaining only PM2.5 measurements, slicing each time series into weekly segments of length 168 (24 hours × 7 days), and assigning season labels based on the calendar month. The combined time series are labeled with two attributes: city (2 levels—2,646 from Beijing and 1,038 from London) and season (4 levels—1,169 winter, 932 spring, 839 fall, and 744 summer). Stratified by city, the dataset is randomly split into 2,579 training, 737 validation, and 368 test samples according to a 70-20-10 ratio.

A.3 NICU Heart Rate

The NICU heart rate dataset consists of 36,679 time series of length 300 (10-minute records sampled every 2 seconds) from 2,780 infants at a Neonatal Intensive Care Unit (2012–2016) [16, 53]. We compute two clinically meaningful attributes for each sequence: heart rate variability (2 levels: low, high) and bradycardia events (2 levels: present or absent). Variability is obtained by dividing each 300-point series into 30 non-overlapping windows of 10 points and taking the median of the window-wise standard deviations. Sequences with values < 1.5 are labeled *low* variability, and those > 3 as *high*. Bradycardia events are identified by detecting contiguous segments where the heart rate drops below 100 bpm for at most 150 time points; a valid event requires a negative drop rate before onset and a positive recovery afterward. Of the 36,679 sequences, 20,000 have low variability and 16,679 have high variability; 2,147 contain bradycardia events. We split the data into 25,675 training, 7,336 validation, and 3,668 test samples, stratified by variability.

B Resolution-wise Semantic Alignment in Contrastive Loss

The contrastive loss (symmetric InfoNCE) between a time series embedding $z_x^{(i)}$ and a instruction embedding $z_c^{(j)}$ is given as:

$$\mathcal{L}_{\text{contrast}} = \frac{1}{2N}\sum_{i=1}^{N}\left[-\log\left(\frac{\exp(z_x^{(i)}\cdot z_c^{(i)})}{\sum_{j=1}^{N}\exp(z_x^{(i)}\cdot z_c^{(j)})}\right) - \log\left(\frac{\exp(z_c^{(i)}\cdot z_x^{(i)})}{\sum_{j=1}^{N}\exp(z_c^{(i)}\cdot z_x^{(j)})}\right)\right],$$

where $z_x^{(i)} \cdot z_c^{(j)}$ denotes the dot-product similarity between the two embeddings. Minimizing this loss increases the numerator $\exp(zx^{(i)}\cdot zc^{(i)})$ and decreases the denominator $\sum_j \exp(zx^{(i)}\cdot zc^{(j)})$, thereby encouraging each matched pair $(zx^{(i)}, zc^{(i)})$ to have higher similarity than any mismatched pair $(zx^{(i)}, zc^{(j)})$ for $j \neq i$.

Now, we divide each embedding into k segments:

$$z_x^{(i)} = [z_{x,1}^{(i)}; \ldots; z_{x,k}^{(i)}], \quad z_c^{(i)} = [z_{c,1}^{(i)}; \ldots; z_{c,k}^{(i)}].$$

The similarity between a pair of embeddings becomes the sum of their segment-wise similarities: $z_x^{(i)} \cdot z_c^{(j)} = \sum_{r=1}^{k} z_{x,r}^{(i)} \cdot z_{c,r}^{(j)}$. Then, the first term in the bracket of $\mathcal{L}_{\text{contrast}}$ (symmetry holds for the second term) can be written as:

$$-\log\left(\frac{\exp(z_x^{(i)}\cdot z_c^{(i)})}{\sum_{j=1}^{N}\exp(z_x^{(i)}\cdot z_c^{(j)})}\right) = -\log\left(\frac{\exp\left(\sum_{r=1}^{k} z_{x,r}^{(i)}\cdot z_{c,r}^{(i)}\right)}{\sum_{j=1}^{N}\exp\left(\sum_{r=1}^{k} z_{x,r}^{(i)}\cdot z_{c,r}^{(j)}\right)}\right)$$

$$= -\log\left(\frac{\prod_{r=1}^{k}\exp(z_{x,r}^{(i)}\cdot z_{c,r}^{(i)})}{\sum_{j=1}^{N}\prod_{r=1}^{k}\exp(z_{x,r}^{(i)}\cdot z_{c,r}^{(j)})}\right) = -\sum_{r=1}^{k}\log\left(\frac{\exp(z_{x,r}^{(i)}\cdot z_{c,r}^{(i)})}{\sum_{j=1}^{N}\prod_{r'=1}^{k}\exp(z_{x,r'}^{(i)}\cdot z_{c,r'}^{(j)})}\right).$$

Minimizing these logarithmic terms will increase the segment-wise similarity $z_{x,r}^{(i)} \cdot z_{c,r}^{(i)}$ while suppressing the similarity between mismatched embedding pairs $z_{x,r}^{(i)} \cdot z_{c,r}^{(j)}$ $(j \neq i)$. Therefore, both the embeddings and their corresponding segments become aligned, linking each segment's semantic representation $(z_{x,r}^{(i)})$ with its corresponding temporal resolution $(z_{c,r}^{(i)})$ during contrastive fusion.

C Natural Language Instruction Generation

The template for each level of each attribute is provided in Table 4. To increase linguistic diversity when evaluating an editor's generalizability to unseen instructions, we use GPT-4o to paraphrase each distinct attribute level with the following prompt:

```
Generate 50 different sentences that have the same meaning
as this sentence related to {context}: '{template sentence}'.
Only return sentences, do not number the sentences, split
the sentences with the character '|' only.
```

Here, {context} denotes the dataset domain, and {template sentence} is the attribute-level description in Table 4. We use temperature 0.1 and split paraphrases 70%/30% for training and held-out evaluation.

D Additional Experiment Results

Across attributes of all three datasets, InstructTime generally achieves optimal editability (ΔDTW ↓, RaTS ↑) and preservability (|RaTS| ↓) under instruction-based settings. A consistent exception is the attribute of local variability (Gaussian-like noise) in the synthetic and NICU datasets, where diffusion-based methods performs better. This suggests that sampling from conditional Gaussian distributions is effective for capturing the varying magnitude of Gaussian noise using prior methods. Incorporating noise estimation mechanisms—such as diffusion models, variational encoder–decoders, or using negative log-likelihood instead of MSE in the loss function—into the InstructTime architecture are important directions for future work.